Le LiDAR : Offrir une vision aux voitures autonomes

Grégoire Fidaire, n° 12989

Objectif recherché

Scanner l’environnement qui entoure le véhicule

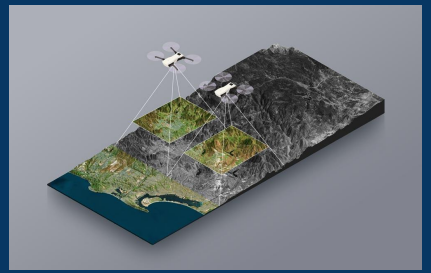
⇒ Assurer la sécurité des passagers

⇒ Reproduire l’environnement le plus réaliste possible

Technologies existantes

Sonar

LiDAR



Photogrammétrie

⇒ **Etude du Sonar pour mon binôme**

⇒ **Etude du LiDAR pour ma part**

⇒ **Abandon de la photogrammétrie car mise en oeuvre complexe et pas adaptée pour une utilisation en temps réel**

Etat de l'art

Critère	LiDAR	Capteur à ultrason
Précision	+++	+
Portée	+++	+
Résolution / Détail	+++	+
Vitesse de mesure	+++	++
Coût	-	+++
Complexité / intégration	+	+++
Sensibilité à la lumière	+	+++

Problématique

Comment le LiDAR et l'ultrason permettent-ils à une voiture autonome de s’adapter aux obstacles ?

Source : generation-nt

Etude du LiDAR

- LiDAR : Light Detection and Ranging - Télédétection par laser
- Première apparition dans les années 1960 et utilisation célèbre avec la mission Apollo 15 qui a cartographié la Lune
- Marché du LiDAR automobile en expansion

Source : generation-nt

Qu'est ce qu'un LiDAR ?

- Repose sur le principe de télémétrie
- Le LiDAR : lasers rayonnant dans le domaine infrarouge, le domaine visible ou le domaine ultraviolet proche (900 nm - 1550 nm)
- Émet une impulsion lumineuse.
- Reçoit l'écho réfléchi sur un obstacle.

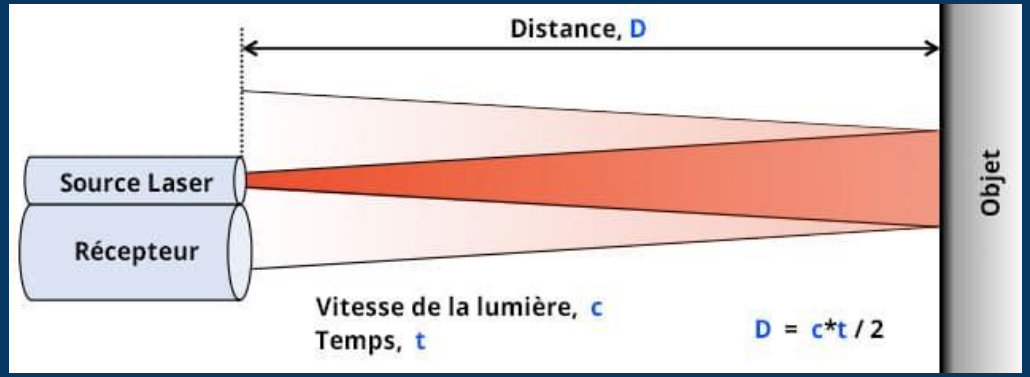


Fig 1 : Schéma explicatif du fonctionnement du LiDAR, Escadrone

Principe de fonctionnement

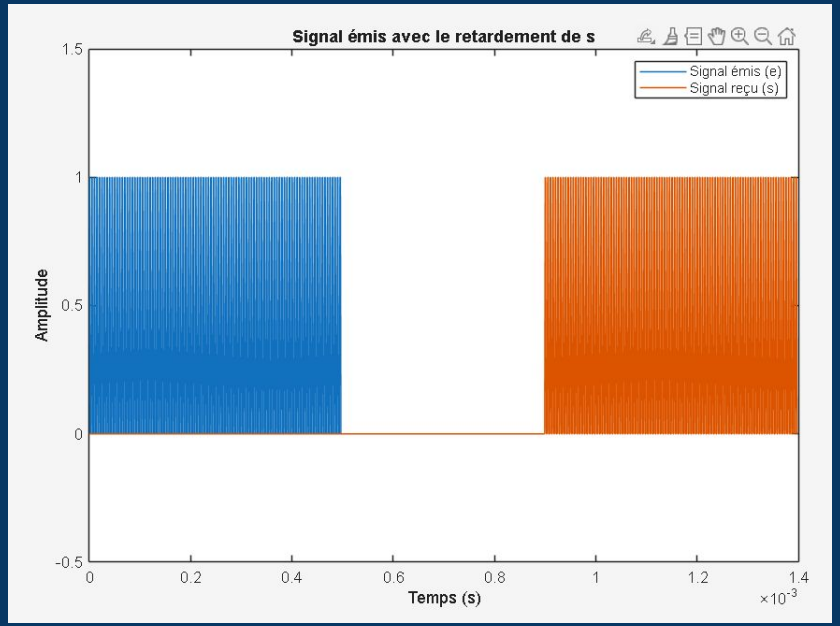


Fig 2 : représentation des signaux originaux sur Matlab

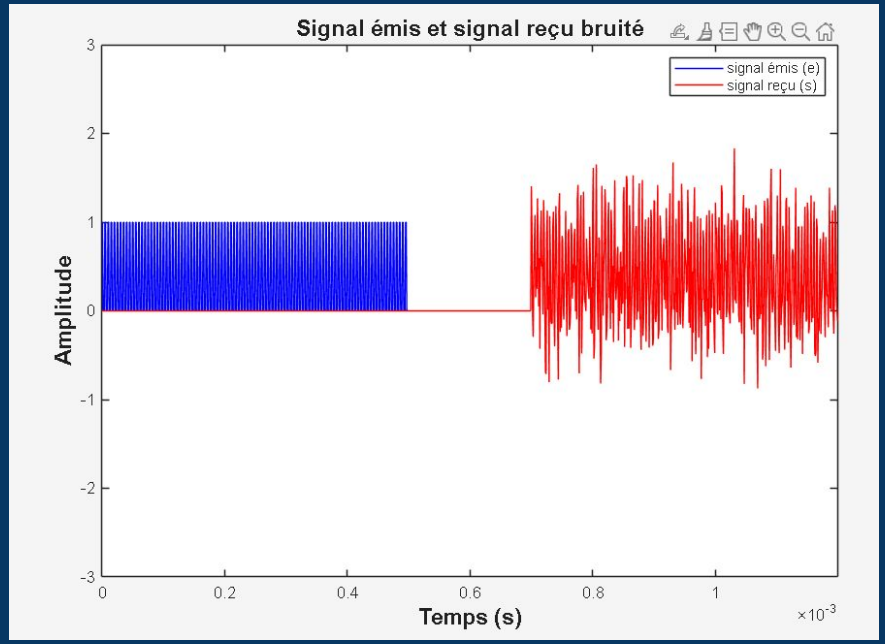


Fig 3 : représentation des signaux originaux avec bruitage sur Matlab

⇒ Vrai temps d'aller retour 400μs pour l'étude

⇒ Choix des seuils et de bruit arbitraires

Principe de fonctionnement

I) Détection par seuil

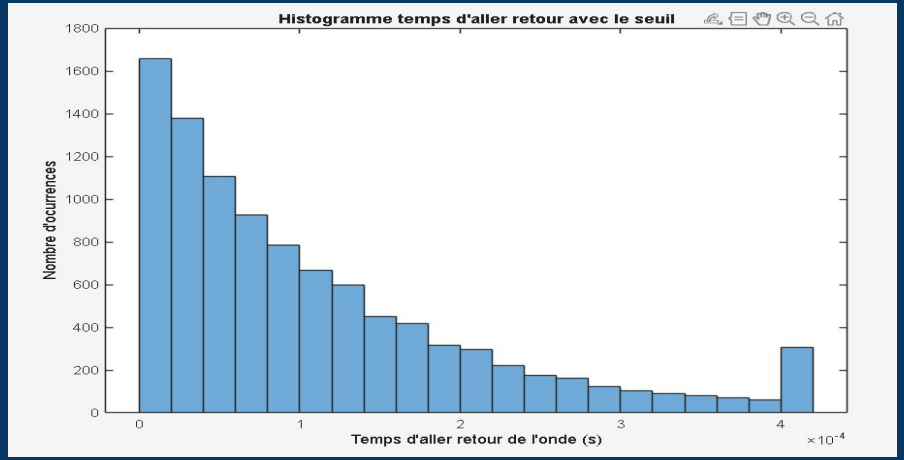


Fig 4 : Représentation de la détection par seuil avec Matlab

⇒ Sensible au bruit
⇒ Résultats très dispersés
⇒ Mauvaise estimation de la distance

II) Détection par corrélation

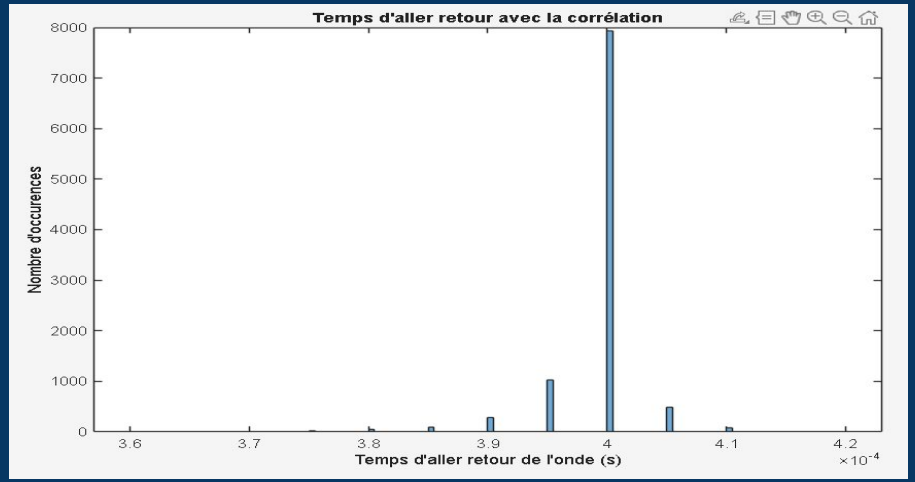


Fig 5 : Représentation de la détection par corrélation avec Matlab

⇒ Robuste au bruit.
⇒ Résultats concentrés autour du vrai temps d'aller retour

Détails en annexe

Caractérisation du type de LiDAR

Analyse du profil du LiDAR sur différents matériaux à 3 mètres :

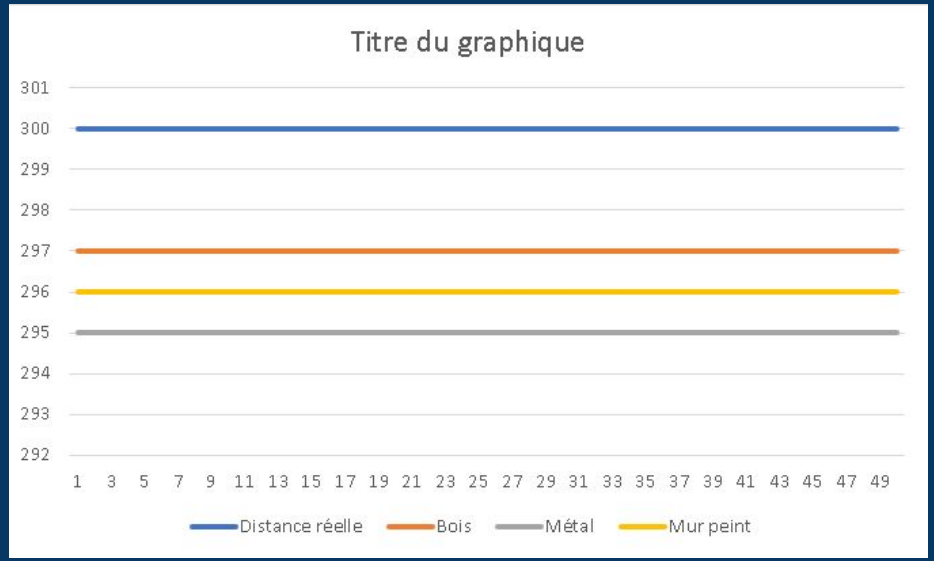


Fig 6 : Comparaison expérimentale entre les matériaux

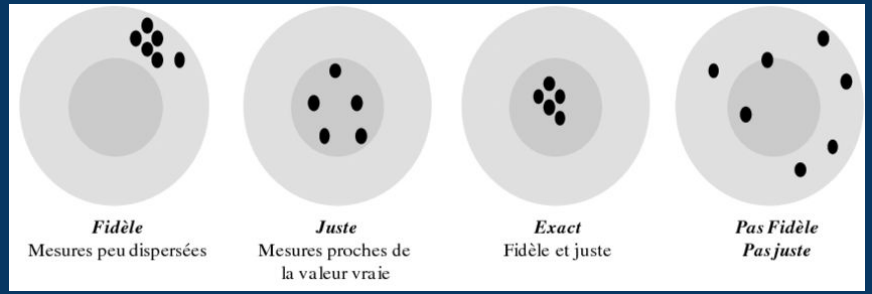


Fig 7 : Principe de caractérisation d'un capteur

⇒ **Données différentes car indice de réfraction différent pour chaque matériau**
⇒ **Potentielle nécessité de filtrer les données reçues**
⇒ **Capteur fidèle**

Précision du LiDAR

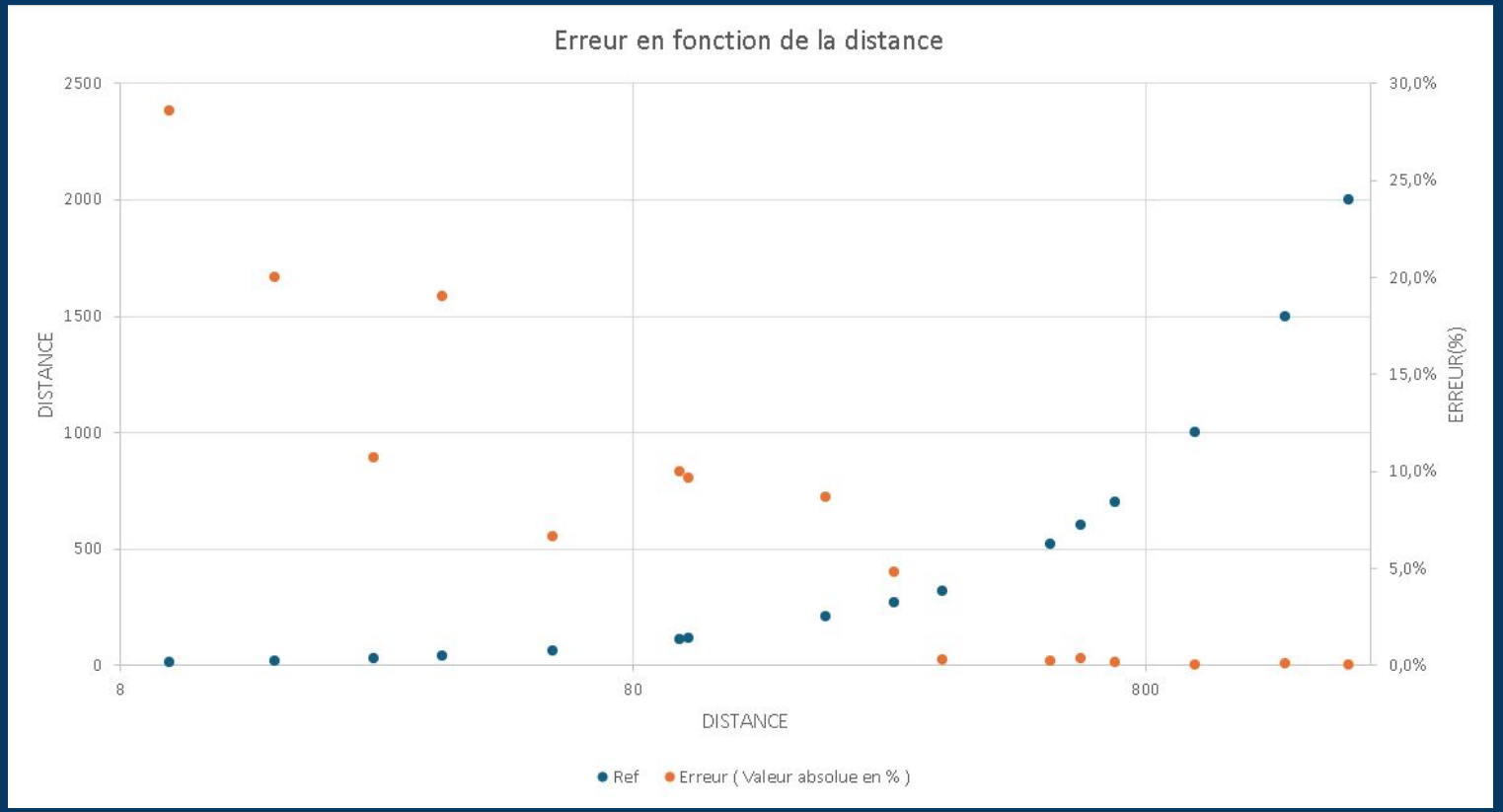


Fig 8 : Courbe représentant l'erreur en absolue en fonction de la distance

Détails en annexe

Maquette expérimentale pour simuler une voiture autonome

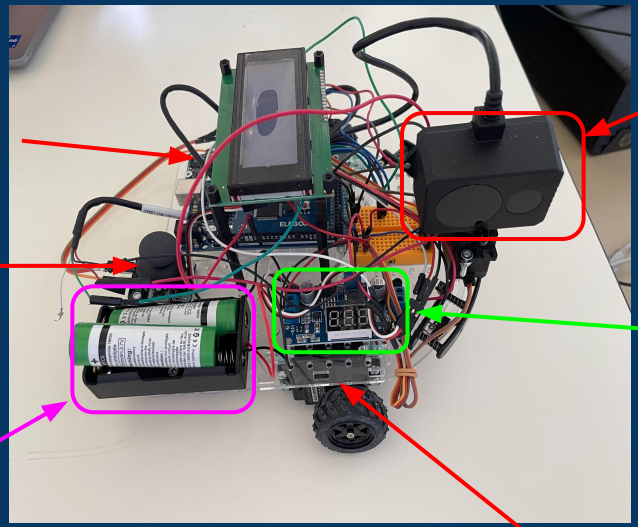


Fig 9 : Maquette Arduino du système

Schéma topo-fonctionnel

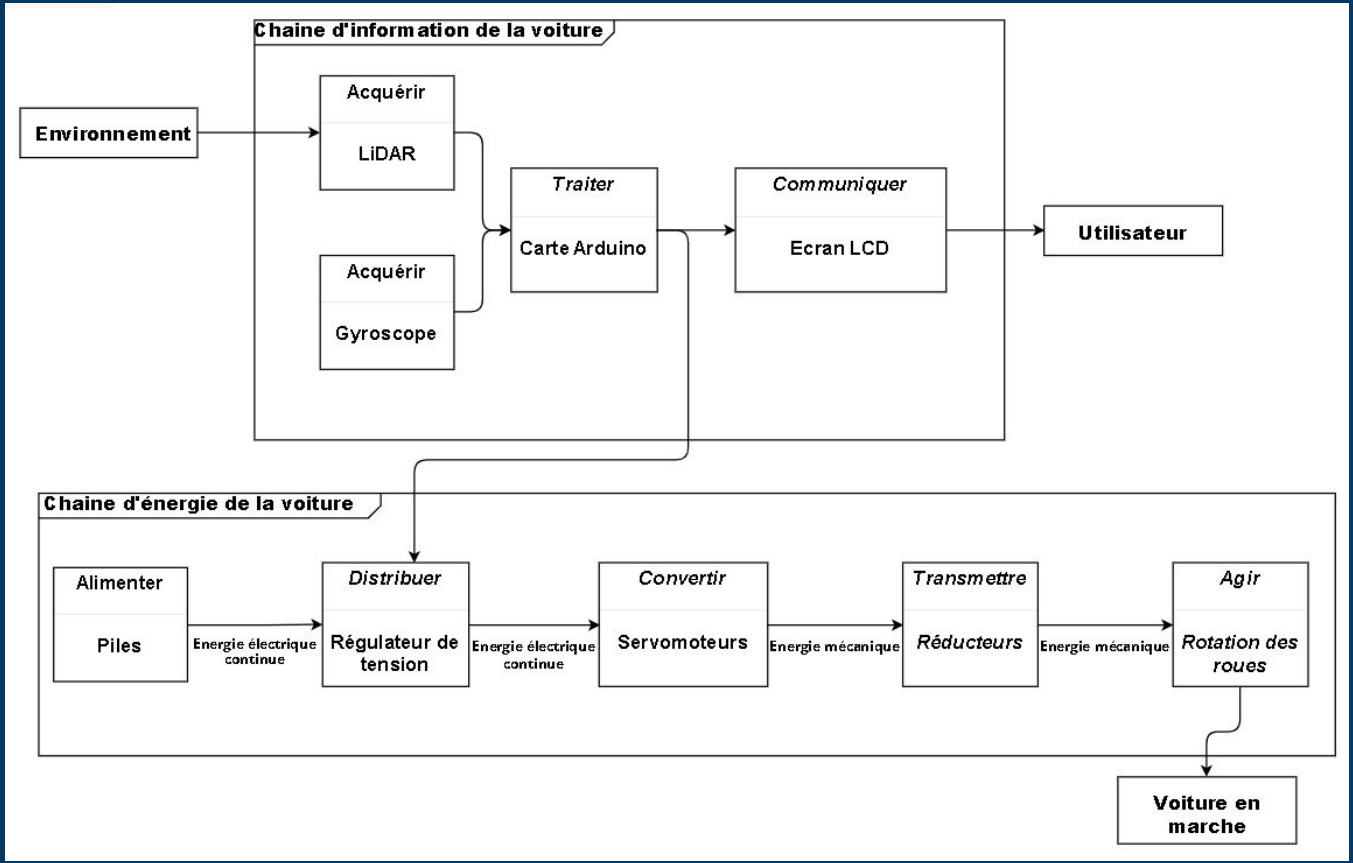


Fig 10 : Schéma topo-fonctionnel du système

II) Etude du LiDAR et du capteur ultrason

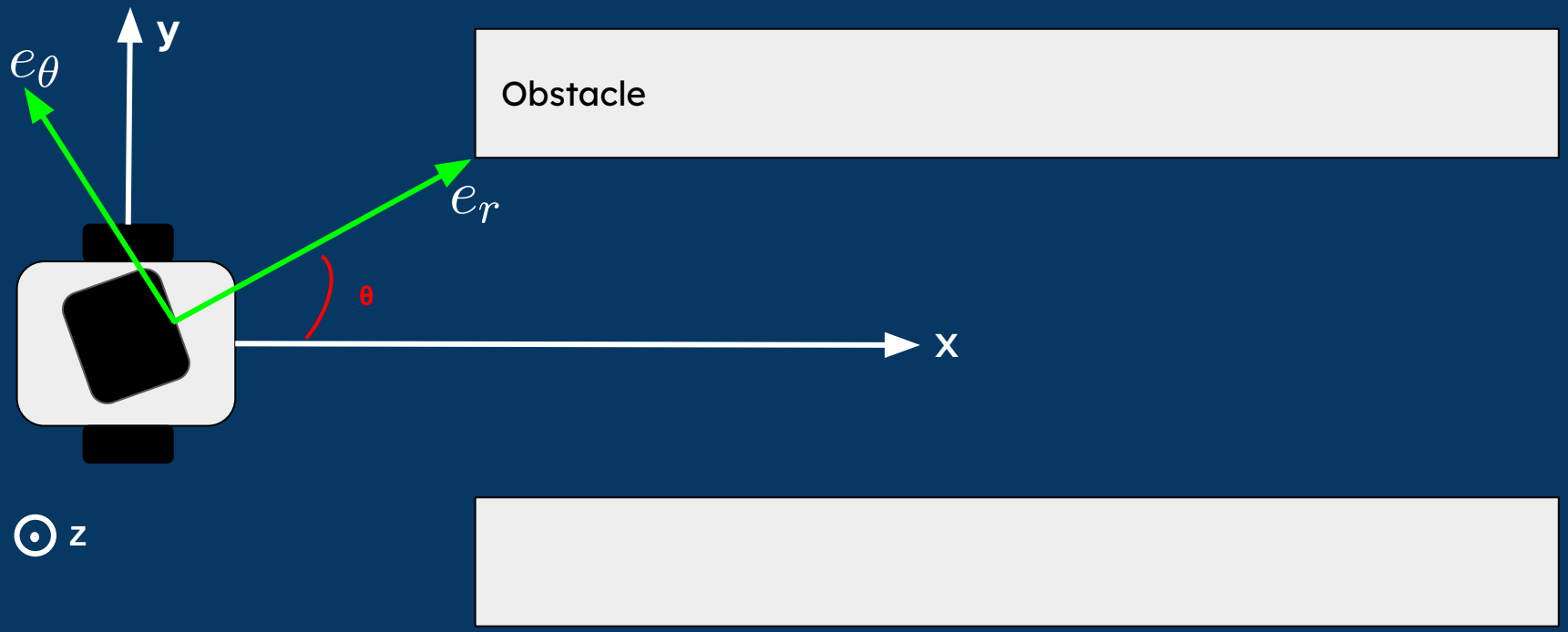


Fig 11 : Représentation initiale prévue

Projection des distances

On a deux bases : $\boldsymbol{B} = (\overrightarrow{x}, \overrightarrow{y})$

$$\boldsymbol{B'} = (\overrightarrow{e_r}, \overrightarrow{e_\theta})$$

Expression des matrices de changement de base :

$$P_{B,B'} = \begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix}$$

$$P^T_{B,B'} = P_{B',B} = \begin{pmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{pmatrix}$$

Partie expérimentale

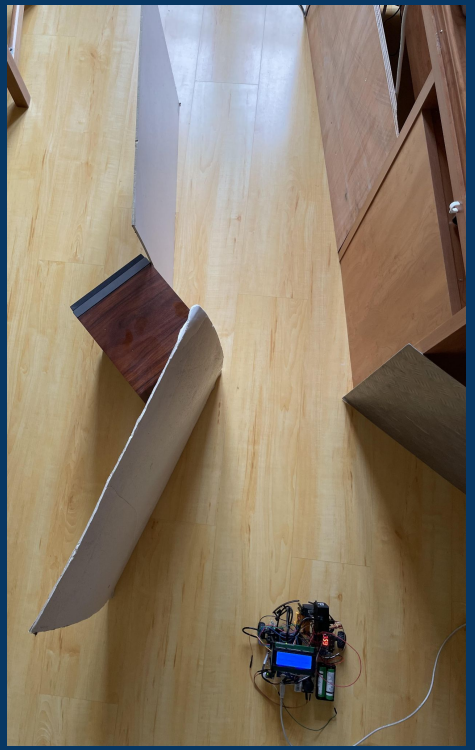


Fig 12 : Chemin choisi pour l'étude

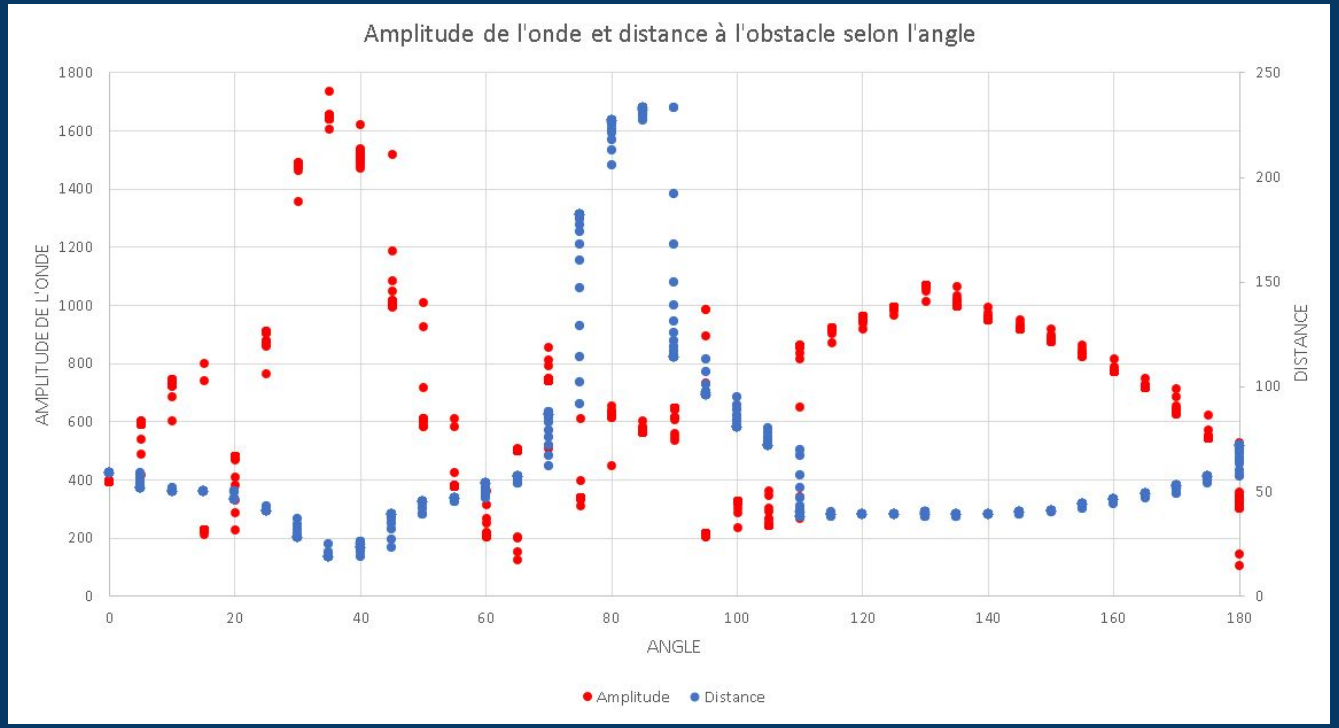


Fig 13 : Données récoltées par le LiDAR avec 50 valeurs par angle avec la distance et l'amplitude de l'onde

⇒ Nécessité de traiter les données pour améliorer la fiabilité des distances

Partie expérimentale

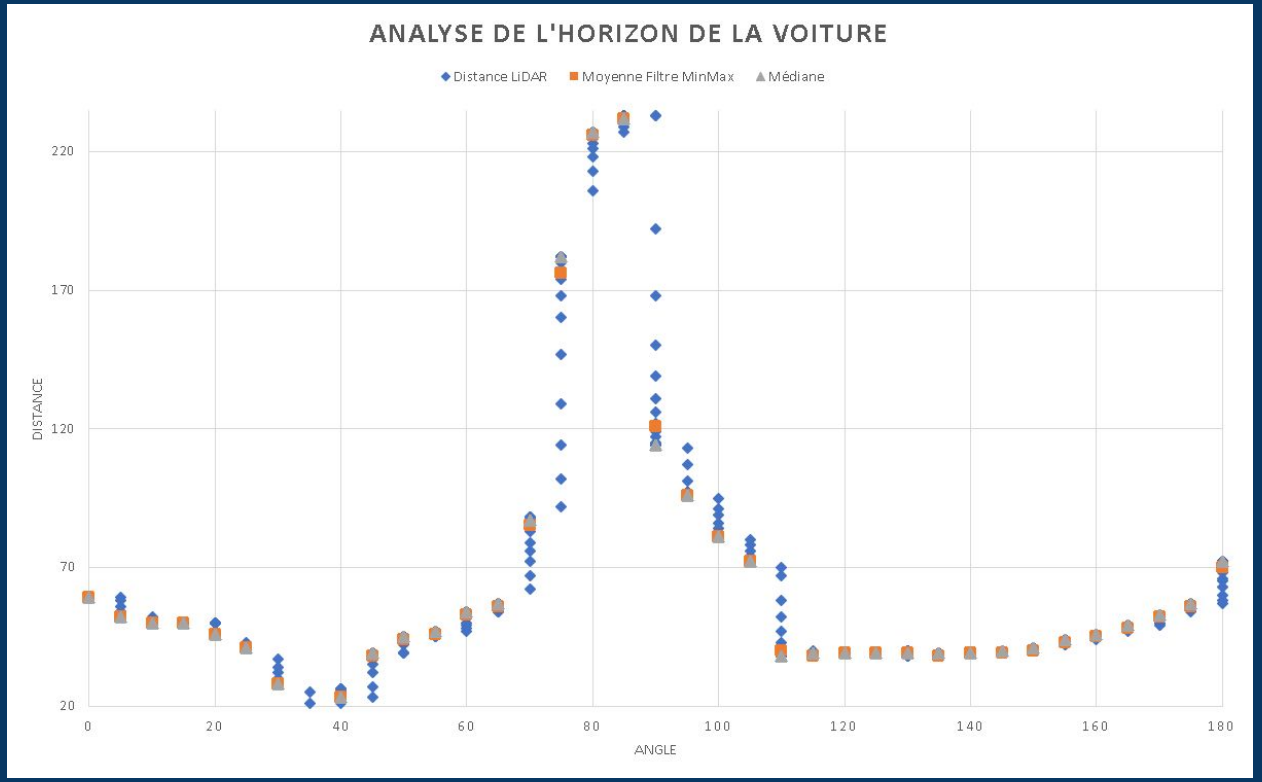


Fig 14 : 50 valeurs par angle avec un filtre éliminant le minimum et le maximum et avec la médiane pour chaque angle

⇒ La moyenne faite avec le filtre est une première approximation

⇒ La médiane apporte plus de fiabilité

Détails en annexe

Caractérisation du type de capteur à ultrason

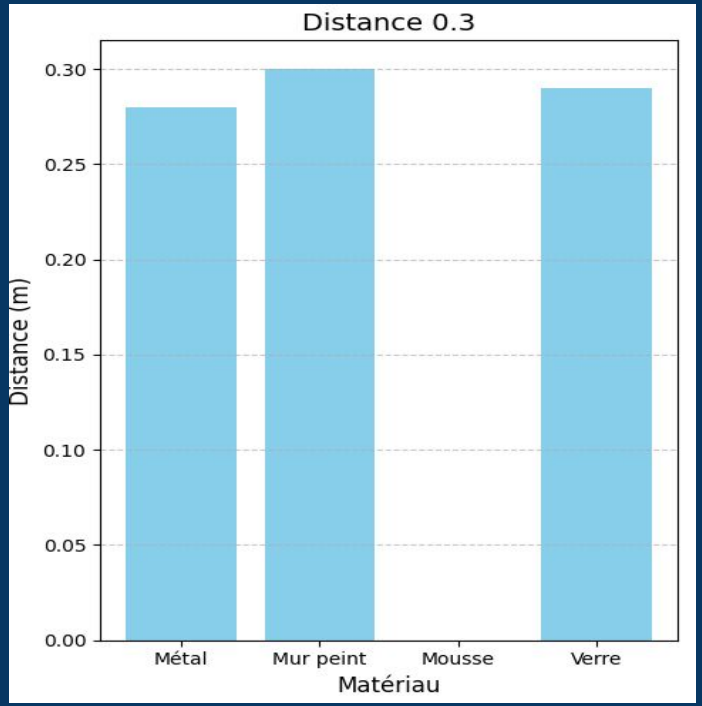


Fig 15 : Distances trouvées par le Sonar selon les matériaux

⇒ Le sonar est un capteur juste

⇒ Indice de réfraction important pour la mousse

Principe de fonctionnement du Sonar

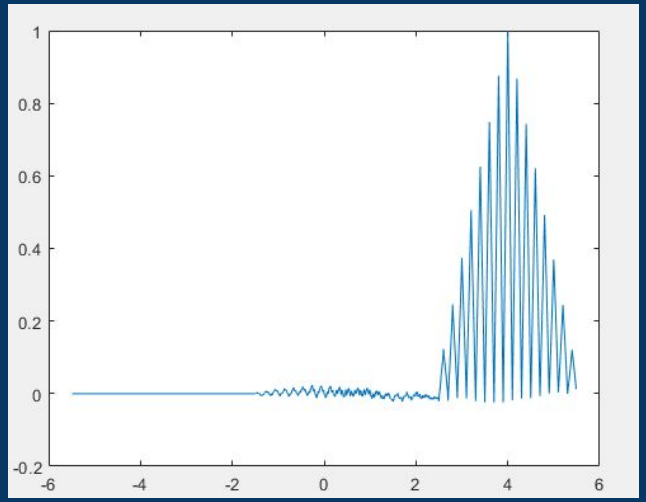
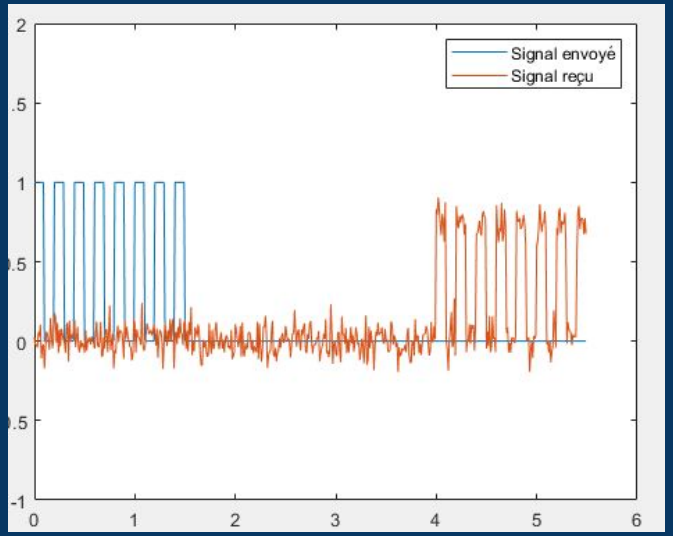


Fig 15 : Détection par corrélation pour le Sonar

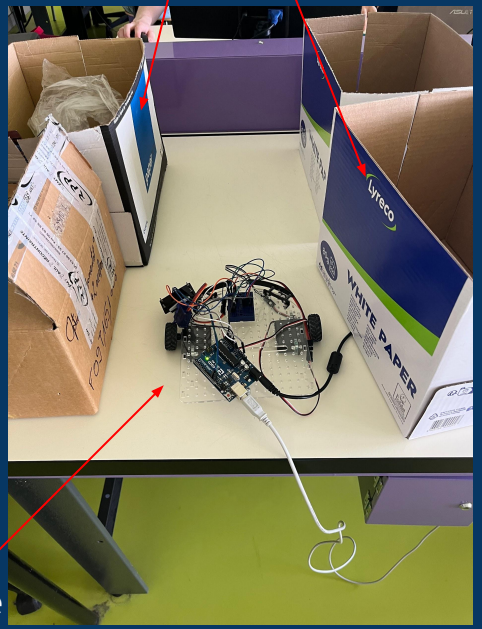
Fig 16 : Représentation du signal émis et bruité pour le Sonar sur Matlab



⇒ Le capteur envoie une salve de 8 ondes

Partie expérimentale du binôme

Murs



Modèle

Fig 17 : Environnement réel d'étude

Modèle

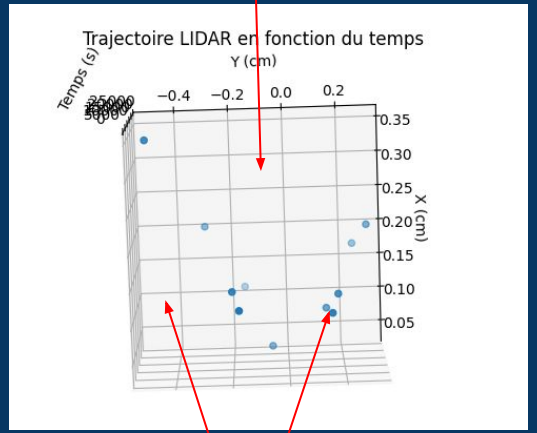


Fig 18 : Environnement numérique donné par le sonar

Murs

Conclusion

⇒ Le LiDAR est une solution fiable à longue distance mais sensible au climat.

⇒Le capteur ultrason est plus adapté à de courtes distances et n’a aucun problème avec les conditions climatiques.

⇒ Pour du temps réel, le LiDAR est la technologie la plus adaptée

Pour aller plus loin

- Approfondir l’étude du LiDAR dans des conditions météorologiques compliquées (pluie, brouillard, …)
- Faire l'étude d’autres technologies
- Mettre en place un modèle réel pour une étude plus approfondis

Annexes

Distance réel	Bois	Métal	Mur peint
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296
300	297	295	296

Set de données pour les matériaux

```
  // Calculer la somme en excluant les valeurs min et max
  int count = 0;
  int countForce=0;
  for (int i = 0; i < repetitions; i++) {
    // Exclure les valeurs min et max
    if (i != minDistanceIndex && i != maxDistanceIndex) {
      totalDistance += tableauDistancesTemp[i];
      count++;
    }
    if (i != minForceIndex && i != maxForceIndex) {
      totalForce += tableauForcesTemp[i];
      countForce++;
      }
  }
  // Calculer la moyenne des valeurs restantes
  if (count > 0) {
    *distance = totalDistance / count;
    *force = totalForce / count;   //
  } else {
    *distance = 0; // Par défaut si aucune donnée valide
    *force = 0;
  }
  if (countForce > 0) {
    *force = totalForce / countForce;
  } else {
    *force = 0;
  }
}
```

Filtre éliminant le min et le max et faisant la moyenne du reste

Annexes

```python
import serial
import time

# Configuration
port = "COM5"
baudrate = 115200
timeout = 1  # secondes
fichier_sortie = "donnees_robot.csv"

try:
    with serial.Serial(port, baudrate, timeout=timeout) as ser, open(fichier_sortie, "w") as fichier:
        print(f" Connecté à {port}")
        print(f" Enregistrement dans : {fichier_sortie}")
        print(" Attente des données série...")

        time.sleep(2)  # ⏰ Laisser le temps à l'Arduino de démarrer

        while True:
            ligne = ser.readline().decode("utf-8", errors="ignore").strip()
            if ligne:
                print(ligne)
                fichier.write(ligne + "\n")
                fichier.flush()

                if ligne.startswith("TOTAL;;"):
                    print("Données complètes reçues. Fin du suivi.")
                    break

except serial.SerialException as e:
    print(f"Erreur de port série : {e}")
except KeyboardInterrupt:
    print("\n Lecture interrompue manuellement.")
```

Code python pour lire le port série et mettre les données dans un fichier csv

Annexes

```matlab
%% Paramètres
te = 1e-6;        % Période d'échantillonnage (1 µs)
ti = 2;           % Largeur d'une impulsion en échantillons
nb_impulsions = 100;
periode_impulsion = 5;                    % Espacement entre impulsions

e = zeros(1, nb_impulsions * periode_impulsion);
for i = 0:nb_impulsions-1
    debut = i * periode_impulsion + 1;
    fin = debut + ti - 1;
    e(debut:fin) = 1;
end
tt = length(e);          % Durée du signal émis

temps = (0:tt-1) * te;

%% Echo reçu sans bruit
tr = 400;                         % retard de 400 échantillons => RTD réel = 400 µs
s = [zeros(1, tr), e];            % Décalage du signal émis
s = s(1:length(e) + tr);          % s est plus long que e

temps_s = (0:length(s)-1)*te;

%% Affichage des signaux (émis et reçu)
figure;
plot(temps, e, 'b');
hold on;
plot(temps_s, s, 'r');
legend('Signal émis', 'Signal reçu');
xlabel('Temps (s)');
ylabel('Amplitude');
title('Signal émis et écho reçu (sans bruit)');
ylim([-0.5 1.5]);

%% Initialisation des vecteurs de résultats
rtd_vecteur = [];
rtd_vecteur_2 = [];
```

```matlab
%% Boucle de simulation avec bruit
for k = 1:10000
    %% Ajout de bruit
    sigma_n2 = 0.1;
    n = sqrt(sigma_n2) * randn(1, length(s));
    s_bruite = s + n;

    %% Détection à seuil (sur signal bruité)
    seuil = 0.75;
    idx = find(s_bruite > seuil, 1);
    if ~isempty(idx)
        rtd = (idx - 1) * te;
    else
        rtd = NaN;
    end

    %% Corrélation
    [v, lags] = xcorr(s_bruite, e);
    v = v / max(v);

    %% Détection du pic de corrélation
    [~, idx_2] = max(v);
    rtd_2 = lags(idx_2) * te;

    %% Sauvegarde des résultats
    rtd_vecteur = [rtd_vecteur, rtd];
    rtd_vecteur_2 = [rtd_vecteur_2, rtd_2];
end

%% Affichage des résultats statistiques
fprintf("RTD réel : %.0f µs\n", tr * te * 1e6);
fprintf("RTD moyen (seuil) : %.2f µs\n", mean(rtd_vecteur, 'omitnan') * 1e6);
fprintf("RTD moyen (corrélation) : %.2f µs\n", mean(rtd_vecteur_2) * 1e6);
```